

TRƯỜNG ĐẠI HỌC XÂY DỰNG HÀ NỘI
BỘ MÔN KHOA HỌC MÁY TÍNH

GIỚI THIỆU VỀ KHOA HỌC MÁY TÍNH

CHƯƠNG 4

Sơ lược về thuật toán

Phạm Hồng Phong

phongph@nuce.edu.vn

Nội dung

- ❖ Một số khái niệm cơ bản
- ❖ Đánh giá độ phức tạp
- ❖ Chứng minh tính đúng
- ❖ Một số nguyên tắc khi trình bày thuật toán

§ 1. Một số khái niệm cơ bản

I. Bài toán (Problem)

- **Ví dụ 1.** Sắp xếp một dãy số

Input: Một dãy số $a = (a_1, a_2, \dots, a_n)$

Output: Dãy $a' = (a'_1, a'_2, \dots, a'_n)$

sao cho a'_i thuộc a và $a'_1 \leq \dots \leq a'_n$

- **Ví dụ 2.** Tìm đường đi ngắn nhất từ u tới v của đồ thị $G=(V,E)$

Input: Đồ thị G và hai đỉnh u, v

Output: Xác định đường đi $d = (u=v_1, v_2, \dots, v_n=v)$ có độ dài ngắn nhất.

§ 1. Một số khái niệm cơ bản

- Bài toán trong tin học: Lớp các bài toán cụ thể cùng loại
- Bài toán được cấu tạo bởi hai thành phần cơ bản:
 - Thông tin vào (input): Cung cấp cho ta các dữ liệu đã có
 - Thông tin ra (output): Những yếu tố cần xác định.
- **Cho bài toán:** cho input và output.
- **Giải bài toán:** từ input, dùng một số hữu hạn các bước thao tác để tìm được output theo yêu cầu của bài toán.

§ 1. Một số khái niệm cơ bản

II. Thuật toán (Algorithm)

Thuật toán là một dãy hữu hạn các thao tác đơn giản được sắp xếp theo một trình tự xác định dùng để giải một bài toán.

Dãy các thao tác đơn giản, có thể “giao cho máy tính làm được” để từ input dẫn ra output.

§ 1. Một số khái niệm cơ bản

- Một số đặc trưng của các thuật toán:
 - Tính tổng quát
 - Tính dừng
 - Tính xác định
 - Tính hiệu quả
- Một số phương pháp diễn đạt (hay mô tả) thuật toán:
 - Liệt kê từng bước
 - Sơ đồ khối
 - Giả mã (pseudo-code)

§ 1. Một số khái niệm cơ bản

Nghiên cứu thuật toán quan tâm đến những vấn đề sau:

- ***Giải được bằng thuật toán***: Lớp bài toán nào giải được bằng thuật toán, lớp bài toán không giải được bằng thuật toán.
- ***Tối ưu hóa thuật toán***: Tìm những thuật toán tốt hơn.
- ***Triển khai thuật toán***: Sử dụng các ngôn ngữ lập trình để thực hiện thuật toán trên máy tính.

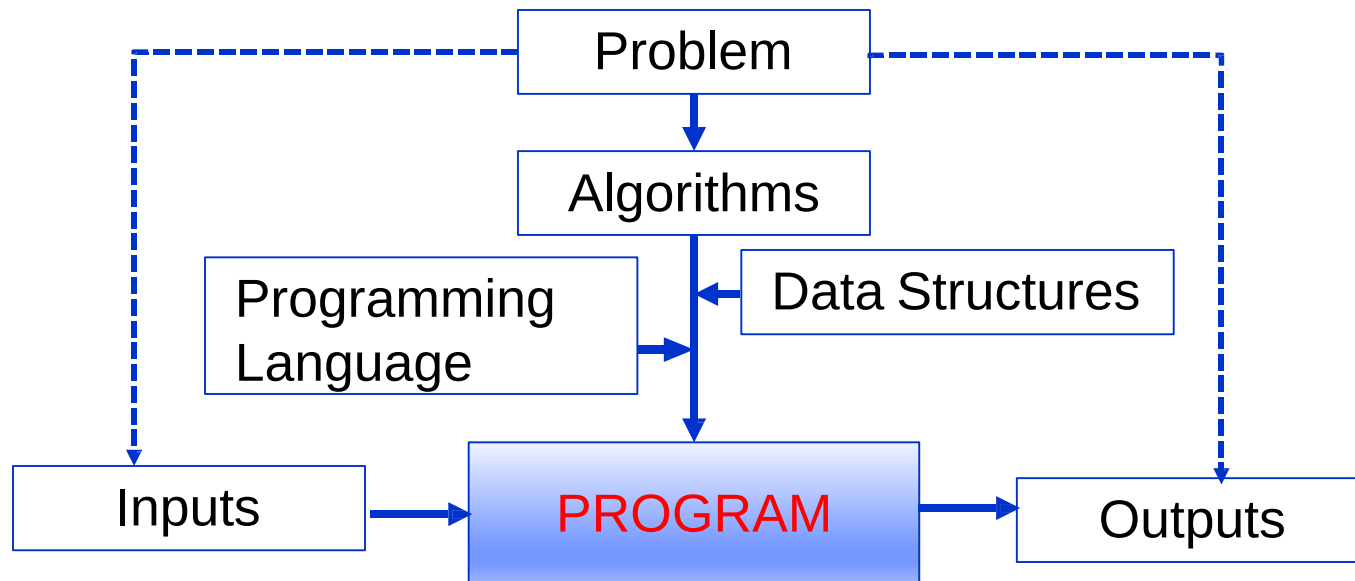
=> **Hai lĩnh vực nghiên cứu chính**

- Thiết kế và đánh giá thuật toán
- Triển khai thuật toán (lập trình).

§ 1. Một số khái niệm cơ bản

III. Chương trình (program)

- Chương trình là sự thể hiện bằng một ngôn ngữ lập trình cụ thể một thuật toán đã cho được xác định trên một cấu trúc dữ liệu xác định.
- Sơ đồ giải bài toán trên máy tính



§ 2. Đánh giá độ phức tạp

- **Phương pháp thực nghiệm:** Lập trình và thử trên các ví dụ
- **Phương pháp lý thuyết:** Tính toán thời gian, bộ nhớ, ... cần thiết của mỗi thuật toán dựa theo độ lớn của input.

Ưu điểm:

- ✓ Không phụ thuộc ngôn ngữ lập trình, loại máy tính
 - ✓ Biết được tính hiệu quả của thuật toán đối với các dữ liệu có kích thước lớn.
- **Thực tế:** Cố gắng sử dụng cả hai phương pháp

§ 2. Đánh giá độ phức tạp

Ký hiệu O (đọc là Ô lớn)

Định nghĩa:

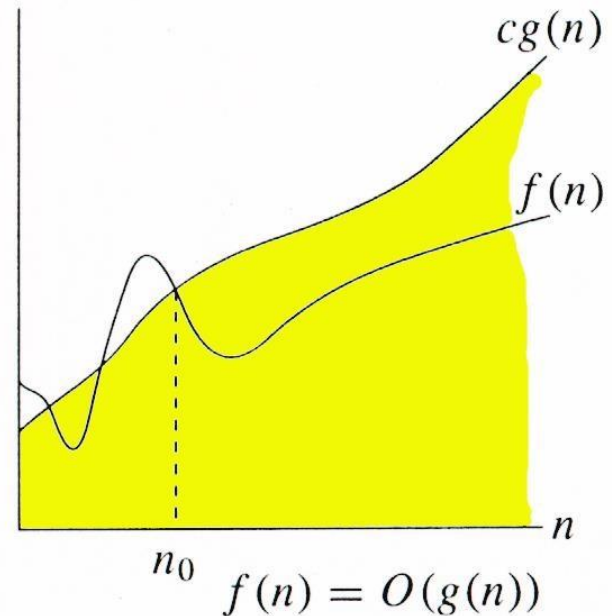
$$f(n) = O(g(n))$$

$\Leftrightarrow$ tồn tại các hằng số c và n_0 để: $f(n) \leq c g(n)$ với mọi $n \geq n_0$

Ví dụ: Xét $T(n) = n^2 + n$

Với mọi $n \geq 1$: $T(n) \leq 2n^2$

Suy ra $T(n) = O(n^2)$



§ 2. Đánh giá độ phức tạp

Nhận xét: Nếu $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = K$ với $K < +\infty$ thì $f(n) = O(g(n))$

Chứng minh: Theo định nghĩa giới hạn của dãy số:

$$\exists n_0 : \left| \frac{f(n)}{g(n)} - K \right| < 1 \text{ với mọi } n \geq n_0$$

$$\text{Lại có } \left| \frac{f(n)}{g(n)} - K \right| \geq \frac{f(n)}{g(n)} - K$$

Suy ra $f(n) \leq (K + 1)g(n)$ với mọi $n \geq n_0$ (đpcm)

§ 2. Đánh giá độ phức tạp

Các quy tắc đánh giá độ phức tạp thuật toán

- Quy tắc bỏ hằng số

Nếu $T(n) = O(c_1 \cdot f(n))$ thì $T(n) = O(f(n))$

Chứng minh:

$$T(n) = O(c_1 \cdot f(n))$$

$$\Rightarrow \text{Tồn tại } c \text{ và } n_0 : T(n) \leq c c_1 \cdot f(n) \text{ với mọi } n \geq n_0$$

Chọn $c' = c c_1$ suy ra:

$$T(n) \leq c' \cdot f(n) \text{ với mọi } n \geq n_0$$

$$\text{Do đó } T(n) = O(f(n))$$

$$\text{Ví dụ: } T(n) = O(2n^2) \Rightarrow T(n) = O(n^2)$$

§ 2. Đánh giá độ phức tạp

▪ Quy tắc cộng

Nếu thuật toán P có hai thành phần liên tiếp P_1 và P_2 với độ phức tạp là $T_1(n) = O(f(n))$ và $T_2(n) = O(g(n))$ thì P có độ phức tạp là $T(n) = O(f(n) + g(n))$.

Chứng minh: Tồn tại các hằng số c_1, n_1, c_2, n_2 sao cho:

$$T_1(n) \leq c_1 \cdot f(n) \text{ với mọi } n \geq n_1$$

$$T_2(n) \leq c_2 \cdot g(n) \text{ với mọi } n \geq n_2$$

Chọn $c = \max(c_1, c_2)$; $n_0 = \max(n_1, n_2)$, ta có:

$$T(n) \leq c(f(n) + g(n)) \text{ với mọi } n \geq n_0 \quad (\text{đpcm})$$

Ví dụ: $T_1(n) = O(n)$; $T_2(n) = O(n^2) \Rightarrow T(n) = O(n + n^2)$

§ 2. Đánh giá độ phức tạp

▪ Quy tắc lấy max

$$T(n) = O(f(n)+g(n)) \text{ và } \lim[f(n):g(n)] = 0 \Rightarrow T(n) = O(g(n)).$$

Chứng minh:

- $T(n) = O(f(n)+g(n)) \Rightarrow T(n) \leq c (f(n)+g(n))$ với mọi $n \geq n_1$
- $\lim[f(n):g(n)] = 0 \Rightarrow f(n) \leq g(n)$ với mọi $n \geq n_2$

Chọn $n_0 = \max(n_1 ; n_2)$:

$$T(n) \leq c (g(n)+g(n)) = 2c \cdot g(n) \text{ với mọi } n \geq n_0$$

Chọn $c' = 2c$:

$$T(n) \leq c' \cdot g(n) \Rightarrow Đpcm$$

Ví dụ: $T(n) = n^4 + 3n^2 + 1 \Rightarrow T(n) = O(n^4)$

§ 2. Đánh giá độ phức tạp

▪ Quy tắc nhân

Nếu thuật toán P được thực hiện bằng cách thực hiện n lần thuật toán P^* .

Giả sử P^* có độ phức tạp là $T_{P^*}(n) = O(f(n))$ thì P có độ phức tạp là $T_P(n) = O(n \cdot f(n))$.

Chứng minh:

$$T_{P^*}(n) \leq c \cdot f(n) \text{ với mọi } n \geq n_0$$

$$\Rightarrow T_P(n) = n \cdot T_{P^*}(n) \leq c \cdot n \cdot f(n) \text{ với mọi } n \geq n_0$$

$$\Rightarrow T_P(n) = O(n \cdot f(n)) \text{ (đpcm)}$$

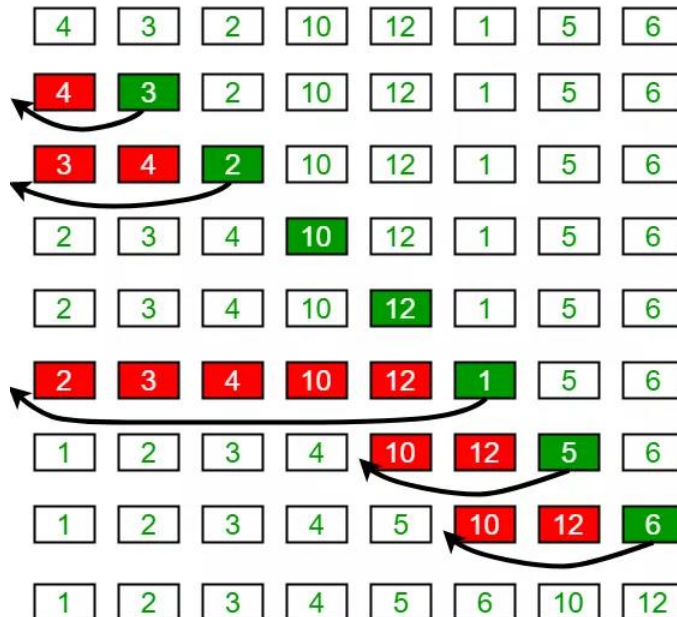
§ 2. Đánh giá độ phức tạp

Ví dụ: Phân tích thời gian thực hiện thuật toán *Sắp xếp chèn*

Ý tưởng: Xuất phát từ bên trái, lần lượt đưa các phần tử kể từ phần tử thứ 2 về đúng vị trí



Insertion Sort Execution Example



§ 2. Đánh giá độ phức tạp

Ví dụ: Phân tích thời gian thực hiện thuật toán *Sắp xếp chèn*

```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```

$O(n^2)$
 $O(1)$
 $O(1)$
 $O(i)$
 $O(1)$
 $O(1)$
 $O(1)$

§ 3. Chứng minh tính đúng

Các chiến lược chứng minh tính đúng (correctness)

- **Kiểm thử (testing):** Chạy thử thuật toán với các dữ liệu vào cụ thể
Ưu điểm: Dễ thực hiện
Nhược điểm: Có thể không phát hiện hết các lỗi
- **Chứng minh tính đúng (correctness proof):** chứng minh bằng toán học
Ưu điểm: Tổng quát
Nhược điểm: Khó hơn và có thể vẫn có lỗi
- **Thực tế:** Cố gắng kết hợp kiểm thử và chứng minh tính đúng

§ 3. Chứng minh tính đúng

Các phương pháp chứng minh tính đúng

- *Đối với thuật toán đệ quy:* Dùng quy nạp
- *Đối với thuật toán không đệ quy:* sử dụng bất biến vòng lặp (loop invariant)

§ 3. Chứng minh tính đúng

Chứng minh tính đúng đối với thuật toán đệ quy

- Sử dụng phương pháp quy nạp: Chứng minh tính đúng của thuật toán theo kích thước dữ liệu vào
 - Cơ sở của quy nạp: Chứng minh tính đúng trong trường hợp đơn giản nhất
 - Quy nạp: Giả sử thuật toán đúng với dữ liệu kích thước n . Chứng minh thuật toán đúng với dữ liệu kích thước $n + 1$



§ 3. Chứng minh tính đúng

- **Ví dụ:** Thuật toán tìm max của một dãy số $A_1, A_2, \dots, A_n$ có n phần tử

Maximum (n) $\equiv$ // Tìm max của dãy số có n phần tử

if ($n=1$) return (A_1)

else return ($\max(\text{Maximum}(n-1), A_n)$);

End.

Chứng minh thuật toán đúng:

- Cơ sở: $n=1 \Rightarrow A_1$ là phần tử duy nhất và lớn nhất
- Quy nạp: Giả sử **Maximum**(n) trả lại giá trị lớn nhất $A_1, \dots, A_n$

Ta có: **Maximum**($n+1$) trả lại $\max(\text{Maximum}(n), A_{n+1}) = \max(A_1, \dots, A_n, A_{n+1})$

$\Rightarrow$ ĐPCM

§ 3. Chứng minh tính đúng

Chứng minh tính đúng đối với thuật toán không đệ quy

Sử dụng bất biến vòng lặp (*loop invariant*)

- Bất biến vòng lặp (*loop invariant*)
 - Bất biến vòng lặp là một khẳng định liên quan đến *các lần lặp*
 - Bất biến đúng trong lần lặp cuối cùng kéo theo thuật toán đúng

§ 3. Chứng minh tính đúng

- Chứng minh tính đúng của thuật toán bằng cách sử dụng bất biến vòng lặp
 - Xác định bất biến vòng lặp sao cho: bất biến vòng lặp đúng ở ***lần lặp cuối cùng*** kéo theo thuật toán đúng
 - Cơ sở: Chứng minh bất biến đúng ở lần lặp đầu tiên
 - Quy nạp: Chứng minh bất biến đúng ở lần lặp trước thì cũng đúng ở lần lặp sau

Quy nạp

§ 3. Chứng minh tính đúng

- **Ví dụ:** Thuật toán tìm max của một dãy số $A_1, A_2, \dots, A_n$ có n phần tử

Maximum (n) $\equiv$ // Tìm max của dãy số có n phần tử

$m = A_1;$

for (i=2; i<=n; i++)

if (m < A_i) $m = A_i;$

return (m)

} Vòng lặp

End.

Ta chứng minh bất biến vòng lặp:

Sau lần lặp ứng với $i = j$ ta có $m = \max(A_1, \dots, A_j)$ (*)

❑ Sau lần lặp với $i = 2$: $m = \max(m, A_2) = \max(A_1, A_2)$

$\Rightarrow$ (*) đúng với $i = 2$

❑ Giả sử sau lần lặp ứng với $i = j$ ta có $m = \max(A_1, \dots, A_j)$

$\Rightarrow$ sau lần lặp ứng với $i = j+1$ ta có $m = \max(m, A_{j+1}) = \max(A_1, \dots, A_j, A_{j+1})$

$\Rightarrow$ ĐPCM

§ 4. Một số nguyên tắc khi trình bày thuật toán

Các bước:

- ☐ Phát biểu bài toán
- ☐ Nêu ý tưởng của thuật toán
- ☐ Ví dụ minh họa
- ☐ Mô tả thuật toán
- ☐ Chứng minh tính đúng
- ☐ Đánh giá độ phức tạp
- ☐ Code
- ☐ Thực nghiệm

Bài tập

Tìm hiểu các thuật toán sắp xếp cơ bản:

- Sắp xếp chèn
- Sắp xếp chọn
- Sắp xếp nổi bọt
- Sắp xếp đổi chỗ trực tiếp